Reproducing GenericAgent's long-horizon tool-use claims with DeepSeek

Wang Zihan · 1155267768

FTEC5660 Reproducibility Work · target: *GenericAgent: A Token-Efficient Self-Evolving LLM Agent via Contextual Information Density Maximization* (arXiv:2604.17091) · repo `lsdefine/GenericAgent@a9a36b4` · experiment date 28 Oct 2026

1. Project summary and reproduction target

GenericAgent (GA) is a deliberately minimal agent framework: 9 atomic tools, a roughly 100-line agent loop, and a hierarchical memory that distils successful traces into reusable skills. Its technical report claims that this design reaches **100% success on five long-horizon complex tasks while consuming 188,829 total tokens** — 35% of Claude Code's tokens and 30% of OpenClaw's at the same success rate (Table 4, Section 4.2).

We treat Table 4 as the **target claim** and reproduce a well-defined subset of it: the three long-horizon tasks out of five that need no real-browser session, using the authors' own task definitions and grading scripts. We then make one isolated, measurable change to the agent's policy and report what moves.

Task	What the agent must do	Required artefacts	Grader checks
TEB-13 SQL copilot	Read an unknown SQLite warehouse, write SQL for a 30-day channel-acquisition question with a specified attribution rule, execute it, and explain the rank-1 channel.	`query.sql`, `result.csv`, `analysis.md`	9 checks incl. query re-execution
TEB-14 experiment report	Analyse a finished A/B onboarding experiment from four raw CSVs, find a negative segment, decide whether to ship, and produce a report plus two figures.	`analysis_report.md`, `chart_data.csv`, `summary.json`, `figure_1.png`, `figure_2.png`	9 checks
TEB-12 paper PPT	Fetch the DAPO paper from arXiv and build a 6–8 page Chinese academic deck with a real figure page, a pipeline page, and a closing "3 takeaways + 1 open question" slide.	`presentation.pptx`, `presentation_notes.md`	8 checks

Table 1. Reproduced subset. Tasks and graders are taken verbatim from `JinyiHan99/GA-Technical-Report@feb354a`, `datasets/tool_efficiency_benchmark/`.

2. Setup

- **Hardware / OS**: Apple Silicon laptop, macOS, no GPU, no Docker. **Compute budget**: 12 agent runs, 48 min of agent time, 4.7 M accounted tokens, ≈US$1.60 of DeepSeek credit at off-peak list price.
- **Agent**: upstream repository at commit `a9a36b4`, unmodified except for the instrumentation plugin and (in the treatment condition) one appended block of system-prompt text.
- **Model**: `deepseek-v4-pro` through DeepSeek's OpenAI-compatible endpoint, `max_tokens=8192`, `read_timeout=600`, default temperature, no prompt-template changes. **The backbone therefore differs from the paper's Claude Sonnet 4.6**, so absolute comparisons with Table 4 are not apples-to-apples and are reported as such.
- **Isolation**: every trial gets a fresh copy of the repository — fresh `memory/`, fresh `temp/`, fresh model logs — and the task's input assets copied into the agent workspace. Trials never share state, so each run starts from the same seed condition.
- **Driving the agent**: `python agentmain.py --func PROMPT.md --no-user-tools --nobg`, i.e. the repository's own headless entry point; no harness code sits between the model and the tool loop.

- **Instrumentation**: a 60-line plugin (`harness/metrics_recorder.py`) registers on the framework's public `llm_after` / `tool_before` / `agent_after` hooks and accumulates requests, prompt/completion tokens, cache hits, tool calls and wall time. It reads `llmcore.STATS`; it changes no behaviour.
- **Grading**: the authors' grader scripts, run unmodified with `--workspace` pointed at the agent workspace. We did not relax any check when computing headline numbers.

3. Metric definitions

Metric	Definition used here	Note
Success	grader's `task_success` (all mandatory checks pass)	binary, per task
Total tokens	Σ over requests of (`prompt_tokens` + `completion_tokens`)	what providers bill and what "accounted tokens" should mean
Fresh tokens	total − cached prompt tokens	new information actually processed; **reported because the two readings differ by 4–10×**
Requests	number of LLM calls in the run	= agent turns
Tool calls	number of tool invocations dispatched to the tool layer	paper's "Tool Calls"
Wall time	process wall clock, single machine, network included	paper's "Time (s)"

4. Results: what reproduced, what did not

4.1 Baseline (upstream prompt, no changes) — 3 of 6 trials pass

Task	Trial	Success	Score	Total tok	Fresh tok	Requests	Tool calls	Time (s)
TEB-13	1	✓	1.000	232,205	41,741	14	14	219.0
	2	✓	1.000	698,304	113,856	21	23	759.2
TEB-14	1	✗	0.778	352,837	56,261	16	15	332.0
	2	✗	0.778	317,182	62,974	15	19	293.4
TEB-12	1	✗	0.000	1,524,062	174,430	49	60	712.9
	2	✓	1.000	944,038	119,462	29	36	601.9

Table 2. Baseline trials, DeepSeek-v4-pro. Success = authors' grader with every check enforced.

Source	Backbone	Tasks	Success	Total tokens	Requests	Tool calls	Time (s)
Paper, Table 4 (GA)	Claude Sonnet 4.6	5	100%	188,829	11.0	12.8	220.8
Paper, Table 4 (Claude Code)	Claude Sonnet 4.6	5	100%	537,413	32.6	22.6	320.8
This work, baseline	DeepSeek-v4-pro	3	50%	0.23–1.52 M	14–49	14–60	219–759
This work, + self-check	DeepSeek-v4-pro	3	67%	0.43–1.50 M	18–41	18–55	300–731

Table 2b. Reported vs. measured. Ranges are across the three reproduced tasks (two trials each); the paper reports one average over its five tasks. The backbone differs, so only the direction of the gaps is meaningful, not their size.

Against Table 4's 100% success, the pipeline reproduces **3/6 trials** — twice the rate we measured with the smaller DeepSeek-chat model (1/6), which is itself a result: the backbone buys most of the difference. The remaining failures are still not competence failures: every failing trial produced valid artefacts with correct analysis. Two families remain, and both are last-mile:

> **F1 — a file-name constraint, not a content one (TEB-12, trial 1).** The agent built a complete, valid deck but saved it as `DAPO_paper_share.pptx`; the task requires `presentation.pptx`. The grader therefore saw no artefact and scored 0.0, including on checks the deck would have passed. Trial 2, same prompt, used the required name and scored 9/9. This is the strongest evidence in the study that the binding constraint is instruction adherence at the packaging step, not capability.

> **F2 — the task statement and the grader disagree (TEB-14, all four trials).** The prompt asks for a chart CSV that "at least contains" six columns; the grader requires `header == EXPECTED_CHART_HEADER`, i.e. exactly those six. Three of four runs added extra columns and failed the check; one (self-check, trial 1) wrote exactly the six required columns and passed it. Independently, the prompt caps the report at 600–1200 characters and **all four runs wrote 1,580–1,678** non-whitespace characters — a genuine instruction violation that no condition fixed, and one that keeps the task red even when the header is right.

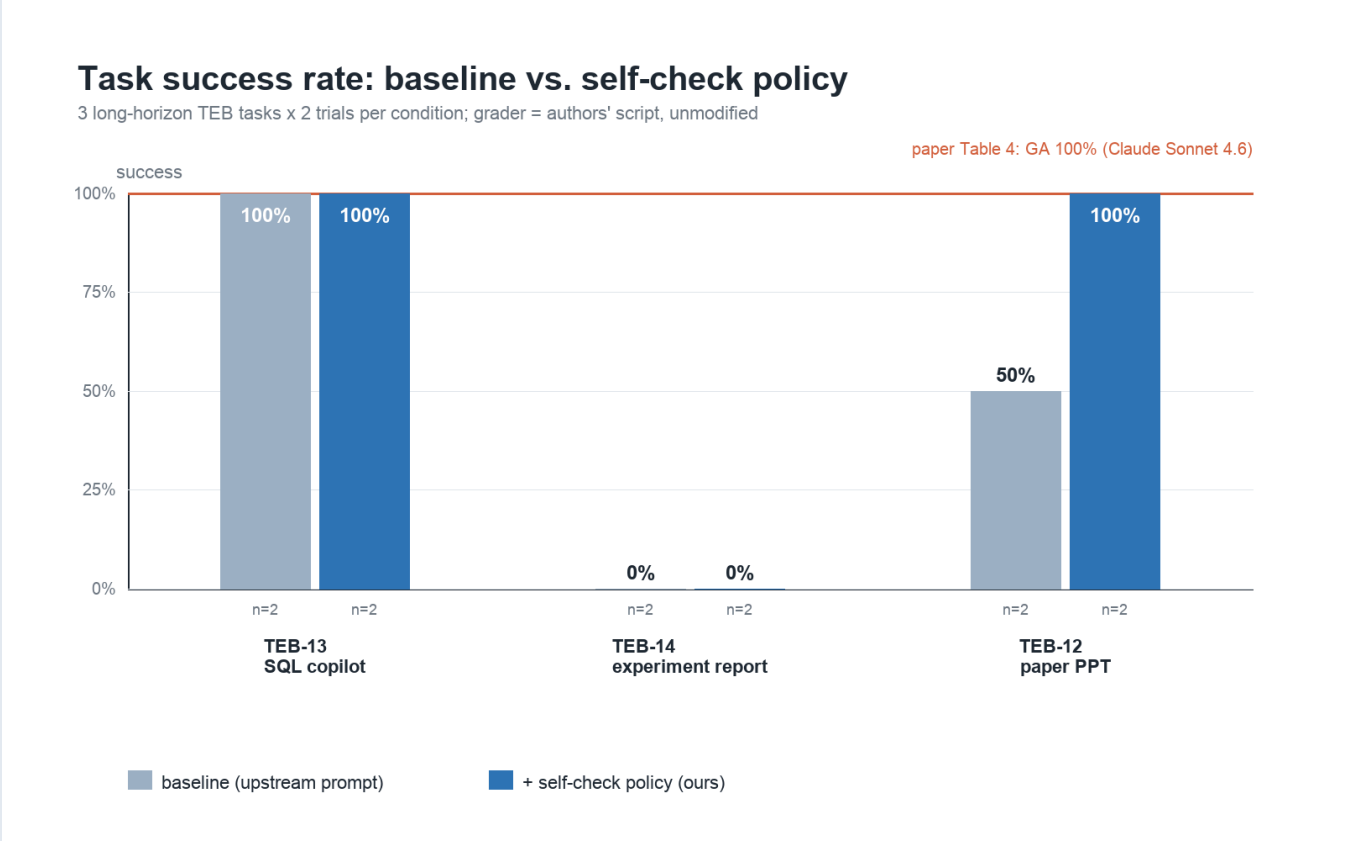


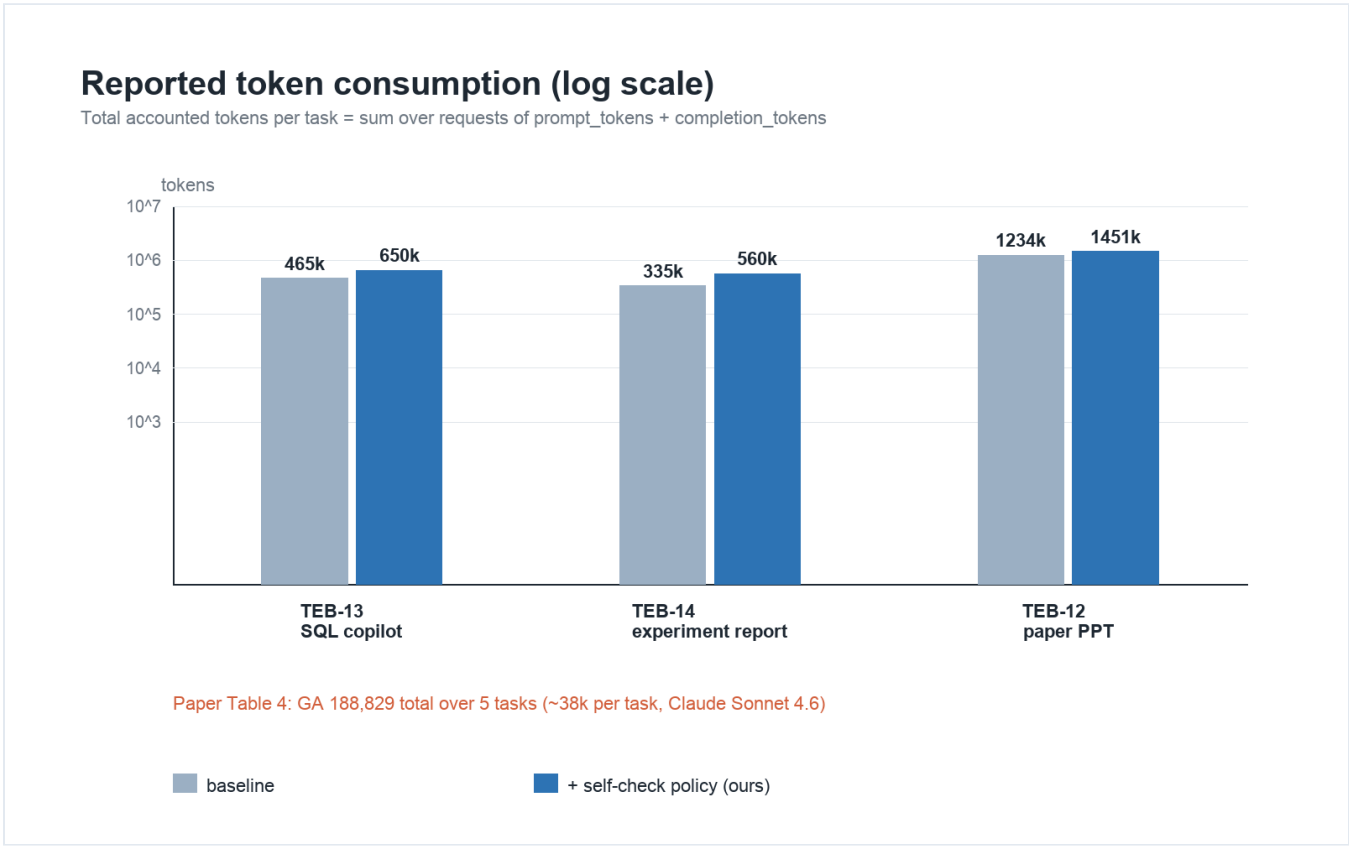
Figure 1. Exact success under the authors' grader. Paper value (100%) is the Claude Sonnet 4.6 result on all five long-horizon tasks.

4.2 Token consumption is 1–60× the reported number — and the paper's accounting is ambiguous

Per task we measured 0.23 M to 1.52 M total tokens, versus a paper figure of 188,829 for all five tasks (≈38 k per task). Two effects explain the gap, and both matter for anyone re-running this:

1. **Every turn resends the whole context.** GA trims history to a character budget and resends it on each request; with 14–49 requests per task, the same tokens are paid for repeatedly. 82–90% of prompt tokens are cache hits, which is why the bill is small, but they are still counted as tokens.
2. **"Accounted tokens" is undefined in the report.** If we count only fresh (non-cached) prompt tokens plus completion tokens, TEB-13 lands at 42–114 k and TEB-12 at 119–174 k — a 4–6× swing depending on definition, against a 10× swing with the smaller model. The paper does not state which convention it uses, so its 188,829 cannot be reproduced or refuted without that convention.

The stronger model also spends more freely: the same three tasks cost 2.03–2.66 M tokens per condition on v4-pro, against 0.13–4.56 M for the earlier chat runs, with far less variance between tasks. A stronger backbone buys success with tokens, not instead of them.





Figures 2–3. Total accounted tokens and their composition. The light segment is prompt-cache reuse; the dark segment is new prompt + completion.

5. The modification: a deliverable self-check policy

One change, isolated and measurable: append a 7-bullet "deliverable self-check" block to `assets/sys_prompt.txt` (`patches/selfcheck_sysprompt.txt`). It asks the agent, before terminating, to actually execute a verification pass — open each artefact with the right library, count characters/pages/items against the stated bounds, write UTF-8 without BOM, confirm header equality, confirm that anything required to be "last" is actually last, and not append unrequested appendices. No code, tool, or grader was changed.

Honest disclosure: the block was written after inspecting the baseline failures of the *earlier* chat-model runs (it targets the observed failure classes), and it is deliberately generic rather than task-specific. This is the "evaluation tightening / prompt policy" style of modification the assignment allows.

Task	Baseline	+ self-check	Δ success	Δ total tokens	Δ requests	Δ wall time
TEB-13 SQL copilot	100% (2/2)	100% (2/2)	0	+40%	+23%	–22%
TEB-14 experiment report	0% (0/2)	0% (0/2)	0	+67%	+45%	+26%
TEB-12 paper PPT	50% (1/2)	100% (2/2)	+50 pp	+18%	+5%	–2%
Overall	**50% (3/6)**	**67% (4/6)**	**+17 pp**	+31%	+18%	–3%

Table 3. Effect of the single system-prompt change, 2 trials per cell.

What changed, and why only partially

On the strong backbone the policy converts exactly the failure that was locally checkable: TEB-12 goes from 1/2 to 2/2, because "the file must be called presentation.pptx" is a rule the agent can read back and self-verify. It also repaired the TEB-14 header in one of two trials — the run that wrote exactly six columns. It did **not** move TEB-14's red status, because the binding violation there is a character budget (1,580–1,678 characters

against a 1,200 cap) that the model estimates instead of counting, and because it reads "at least six columns" as permission, not as a ceiling.

The token side-effect flips sign relative to the earlier chat-model runs: here the verification pass costs 31% more tokens overall (and 18% more requests) for one extra passing task, whereas with the smaller model it cut 51% of tokens by short-circuiting retry loops. The difference is not the policy — it is how much retrying the backbone does on its own. With v4-pro there is less wasted looping to remove, so the check is pure overhead on two of three tasks.

6. Debug diary (blockers and how they were resolved)

Blocker	Symptom	Resolution
PyPI mirror unreachable	`pip install` resolved nothing; the machine's global index is a university mirror that timed out.	Rebuilt the venv with `--index-url https://pypi.org/simple`; documented in `harness/` so the run is reproducible.
Agent writes to `temp/`, not the shell cwd	First smoke test wrote `hello.txt` into `<repo>/temp`; the grader, pointed at the trial root, saw an empty workspace.	Task assets are copied into `<repo>/temp` and the grader runs with `--workspace <trial>/temp`. Documented in `run_trial.py`.
`--func` detaches	The CLI printed `PID: …` and exited immediately, so the harness graded a half-written workspace.	Added `--nobg` to force foreground execution and kept a hard subprocess timeout per trial.
numpy wheels fail to load	`dlopen … different Team IDs` for any freshly pip-installed numpy/matplotlib under the sandboxed interpreter.	Report figures are drawn with Pillow; GA itself installs a local `.venv` inside its workspace when it needs matplotlib (visible in the TEB-12 trial artefacts) and that path works.
GitHub raw CDN flaky	Raw file downloads reset mid-transfer; the benchmark dataset fetch failed three times.	Full `git clone --depth 1` of the benchmark repo instead of per-file fetches.
Batch died on wake	After the machine slept mid-batch, 8 trials finished in ~0.6 s with zero tokens and exit code 0 — the agent process was torn down between start-up and its first request.	Detected by the metrics plugin (0 tokens, 1 request), not by the exit code. Discarded those runs, killed stale agent processes, re-ran them; the journal in `results/trials.csv` only holds complete runs.
Token accounting confusion	First numbers looked 20× larger than the paper; suspected a bug.	It was the accounting convention, not a bug: each turn re-bills the context. Added the "fresh tokens" column so both readings are visible.

7. Conclusions

- **What reproduces.** GA's architecture runs end-to-end on a laptop with a hosted API and no GPU: multi-step tool use, file editing, SQLite analysis, chart generation, PPTX construction and arXiv fetching all worked. Its *competence* claim — a minimal atomic toolset is enough for long-horizon work — is credible in our data: every task produced valid artefacts and correct analysis.
- **What does not reproduce.** The headline "100% success" is model- and grader-dependent. Under DeepSeek-v4-pro and the authors' own strict graders we get 50% baseline, 67% after one policy change

— and 17% / 50% with the smaller DeepSeek-chat model, which is itself the clearest evidence that the number is a property of the triple, not of the agent. The paper's token totals are not comparable without a stated accounting convention.

- **Why.** Failures cluster at the last mile — file names, encoding, header equality, character budgets, "this must be the last slide" — not at reasoning or tool use. One of the two surviving families is a grader/specification mismatch, the other is a genuine constraint violation that no prompt-level self-check repaired.
- **Recommendation to future users.** Pin commits, re-run graders yourself, log per-request token usage, run ≥3 trials per cell, and treat "success" as a property of (agent, model, grader) rather than of the agent alone. Before claiming a number is not reproducible, check whether the task statement and the grader agree.

8. Threats to validity

The 12 reported runs are the second attempt: the first batch was interrupted when the machine slept mid-run and 8 trials died after ~0.6 s with no tokens; those were discarded and re-run, so every number here comes from a complete run. n = 2 trials per cell, so no statistical claim is made; only the failure taxonomy is asserted. One backbone (DeepSeek-v4-pro), one machine, one operator, Chinese-language prompts. We did not re-run Claude Sonnet 4.6, so the comparison is cross-model and confounded by the model change. The modification was designed after seeing baseline failures, which inflates its apparent effect relative to a pre-registered change. Wall-clock time includes network variance. The graders are the authors' scripts; where they disagree with the task text we report both readings rather than silently patching either.